# Interface summary 

31 interfaces and 10 type aliases from 10 files, in 7 groups of related interfaces. Inherited members are listed under the interface that declares them. Generated 2026-11-11 by npm run summary.

## RectangleOps group

### Point geometry.ts

*A location, in the coordinate space its use states.*

**Used by** RectangleOps; Shape, InkStroke, Circle, ShapeLocatorQueries (page 2); EventDistributor (page 3).

```
readonly x: number
readonly y: number
```

### Rectangle geometry.ts

*An axis-aligned rectangle covering the half-open region [x, x + width) × [y, y + height).*

**Used by** RectangleOps; Shape, ShapeLocatorQueries (page 2); LayoutObject (page 3).

```
readonly x: number
readonly y: number
readonly width: number
readonly height: number
```

### RectangleOps geometry.ts

*Operations on rectangles, following Rectangle's half-open semantics. cut, splitHoriz, splitVert, inset and place are each defined exactly by their reference functions in geometry.vocab.ts.*

```
contains(r: Rectangle, p: Point): boolean
```
Whether p lies in r.

```
intersects(a: Rectangle, b: Rectangle): boolean
```
Whether a and b overlap.

```
cut(r: Rectangle, edge: Edge, size: number): [Rectangle,
  Rectangle]
```
Cuts a strip of the given size off one edge of r.

```
splitHoriz(r: Rectangle, ...weights: number[]):
  Rectangle[]
```
Splits r left to right into one rectangle per weight, each as wide as its share of r's width, and as tall as r. splitHoriz(r, 5, 10, 5) gives a quarter, a half and a quarter.

```
splitVert(r: Rectangle, ...weights: number[]):
  Rectangle[]
```
Splits r top to bottom into one rectangle per weight, each as tall as its share of r's height, and as wide as r.

**RectangleOps, continued**

```
inset(r: Rectangle, d: number): Rectangle
```
r shrunk by d on every side.

```
place(r: Rectangle, size: { readonly width: number;
  readonly height: number }, alignX: Align, alignY:
  Align): Rectangle
```
A rectangle of the given size placed within r, aligned along each axis at the start, the center (rounded down to a whole unit) or the end.

### Viewport geometry.ts

*A 2D similarity transform (uniform scale, rotation, translation) from canonical coordinates to view coordinates (see the top of this file).*

**Used by** View (page 4).

```
readonly scale: number
```
View units per canonical unit.

```
readonly rotation: number
```
Radians.

```
readonly tx: number
readonly ty: number
```

### Types

```
type Edge = "top" | "bottom" | "left" | "right"
```
A side of a rectangle.

**Used by** RectangleOps.

```
type Align = "start" | "center" | "end"
```
Where a smaller rectangle goes within a larger one, along one axis.

**Used by** RectangleOps.

# Interface summary 

31 interfaces and 10 type aliases from 10 files, in 7 groups of related interfaces. Inherited members are listed under the interface that declares them. Generated 2026-11-11 by npm run summary.

## ShapeLocator group

Refers to Point, Rectangle (page 1).

### Color color.ts

*An sRGB color with straight (not premultiplied) alpha.*

**Used by** InkStroke, Circle.

```
readonly r: number
readonly g: number
readonly b: number
readonly a: number
```

### Shape shape.ts

*A region of the plane, such as an ink stroke.*

**Used by** InkStroke, Circle, ShapeLocator; AddShape (page 3); ShapeCollection, ShapeCollectionWriter, ShapeMessageProcessor (page 6).

```
readonly kind: string
```
Identifies the shape's type.
```
boundingBox(): Rectangle
```
The exact extents of the shape: x and y are its minimum x and y, and x + width and y + height are its maximum x and y.
```
intersects(r: Rectangle): boolean
```
Whether at least one point lies both in this shape and in r.
```
contains(p: Point): boolean
```
Whether p lies in the shape.

### InkStroke extends Shape shape.ts

*A stroke of digital ink: a polyline drawn with a round pen.*

**Extends** Shape.

```
readonly kind: "ink"
readonly points: readonly Point[]
```
The polyline's vertices, in drawing order.
```
readonly thickness: number
```
Pen diameter, in canonical units.
```
readonly color: Color
```

### Circle extends Shape shape.ts

*A filled circle: the points within radius of center, with the half-open rule at its maximum x and y.*

**Extends** Shape.

```
readonly kind: "circle"
readonly center: Point
readonly radius: number
```
In canonical units.
```
readonly color: Color
```

### ShapeLocatorQueries shape.ts

*Finds shapes by location.*

**Used by** ShapeLocator; ShapeCollection (page 6).

```
inRect(r: Rectangle, out: ShapeId[]): number
```
Finds the shapes that intersect r.
```
atPoint(p: Point, out: ShapeId[]): number
```
Finds the shapes that contain p.

### ShapeLocator extends ShapeLocatorQueries shape.ts

*A spatial index that finds shapes by location, such as a quadtree.*

**Extends** ShapeLocatorQueries.

```
add(id: ShapeId, shape: Shape): void
```
Indexes shape under id.
```
remove(id: ShapeId): boolean
```
Removes id from the index.
```
removeMultiple(ids: readonly ShapeId[]): number
```
Removes every listed ID that is present. ids may contain duplicates and IDs that aren't present; both are ignored.

### Types

```
type ShapeId = string & { readonly __brand: "ShapeId" }
```
Identifies a shape on every client.

**Used by** ShapeLocatorQueries, ShapeLocator, ShapeId; AddShape, RemoveShapes (page 3); ShapeCollection, ShapeCollectionWriter, ShapeMessageProcessor (page 6).

# Interface summary 

31 interfaces and 10 type aliases from 10 files, in 7 groups of related interfaces. Inherited members are listed under the interface that declares them. Generated 2026-11-11 by npm run summary.

## Operation group

Refers to ShapeId, Shape (page 2).

### Operation dataModel.ts

*One change to a data model.*

**Used by** AddShape, RemoveShapes, ClearShapes; OperationSource, MessageProcessor (page 5).

```
readonly type: string
readonly clientId: string
readonly clientSeq: number
readonly refSeq: number
seq: number
```

### RemoveShapes extends `Operation` shapeModel.ts

*Removes the listed shapes that are present.*

**Extends** Operation.

```
readonly type: "remove"
readonly ids: readonly ShapeId[]
```

### ClearShapes extends `Operation` shapeModel.ts

*Removes the shapes its client knew about (refSeq in dataModel.ts).*

**Extends** Operation.

```
readonly type: "clear"
```

### AddShape extends `Operation` shapeModel.ts

*Adds shape under id.*

**Extends** Operation.

```
readonly type: "add"
readonly id: ShapeId
readonly shape: Shape
```

## LayoutObject group

Refers to Rectangle, Point (page 1).

### LayoutObject layout.ts

*Something placed in a region of a window.*

**Used by** EventDistributor.

```
region: Rectangle
```

Where it is, in the window's coordinates.

### EventDistributor<M> extends `LayoutObject` layout.ts

*Passes a platform's positional events to listeners by where they fall.*

**Extends** LayoutObject.

```
locate(event: M[keyof M]): Point
```

Where event happened, in the window's coordinates.

```
on<E extends keyof M & string>(eventName: E, listener:
   Listener<M, E>, target: LayoutObject): void
```

Registers listener for eventName on target.

```
off<E extends keyof M & string>(eventName: E, listener:
   Listener<M, E>, target: LayoutObject): void
```

Removes the most recent registration of listener for eventName on target; does nothing if there is none.

```
distribute<E extends keyof M & string>(eventName: E,
   event: M[E], except?: LayoutObject): boolean
```

Passes event to the innermost target, among those registered for eventName other than except, that contains it.

### Types

```
type Listener<M, E extends keyof M> = (eventName: E,
   event: M[E]) => void
```

Gets one of a platform's events: its name, and the platform's event object.

**Used by** EventDistributor.

# Interface summary 

31 interfaces and 10 type aliases from 10 files, in 7 groups of related interfaces. Inherited members are listed under the interface that declares them. Generated 2026-11-11 by npm run summary.

## FrameHandler group

Refers to Viewport (page 1); ShapeCollection (page 6).

### EventEmitter<Events extends EventMap> events.ts

*Something that emits events.*

**Used by** View; DataModelReader, OperationSource (page 5).

`on<E extends keyof Events & string>(event: E, listener: (...args: Events[E]) => void): this`

Registers listener for event.

`off<E extends keyof Events & string>(event: E, listener: (...args: Events[E]) => void): this`

Removes the most recent registration of listener for event; does nothing if it has none.

### FrameHandler frames.ts

*Something that does its work during a display frame.*

**Used by** FrameScheduler, View.

`frame(timeMs: number): void`

timeMs is the frame's timestamp in milliseconds, on the window's monotonic clock.

### View<M extends ViewModel> extends FrameHandler, EventEmitter<{ viewport: [Viewport] }> view.ts

*Shows one model through a viewport, and stays current.*

**Extends** FrameHandler, EventEmitter. **Used by** ShapeView.

`readonly model: M`

`viewport: Viewport`

Setting this redraws the view at the next frame and emits viewport with the new value.

`dispose(): void`

Ends the view: removes its listener from the model, and requests, draws and emits nothing more.

### ShapeView extends View<ShapeCollection> shapeView.ts

*A view of shapes.*

**Extends** View.

`frame(timeMs: number): void`

Draws the model as it is now.

### FrameScheduler frames.ts

*Calls handlers before the window's next display refresh.*

`requestFrame(handler: FrameHandler): void`

Asks for one call to handler.frame() before the next display refresh.

## Types

`type EventMap = Record<string, unknown[]>`

Each event's name, mapped to the arguments its listeners get.

**Used by** EventEmitter; ModelChangeEvents (page 5).

`type ViewModel = DataModelReader<{ change: [] }>`

A model a view can show: its read side, which emits change after each change.

**Used by** View.

# Interface summary 

31 interfaces and 10 type aliases from 10 files, in 7 groups of related interfaces. Inherited members are listed under the interface that declares them. Generated 2026-11-11 by npm run summary.

## OperationSink group

Refers to Operation (page 3); EventMap, EventEmitter (page 4).

### ModelChangeEvents extends `EventMap` dataModel.ts

*A model's change events.*

**Used by** DataModelReader.

### DataModelReader`<E extends ModelChangeEvents>` extends `EventEmitter<E>` dataModel.ts

*The read side of a data model.*

**Extends** EventEmitter (page 4). **Used by** MessageProcessor; ShapeCollection (page 6).

### OperationSink`<T>` dataModel.ts

*Where producers put operations.*

**Used by** OperationQueue, MessageProcessor.

`enqueue(item: T): void`

### OperationQueue`<T>` extends `OperationSink<T>` dataModel.ts

*A queue of operations, read by the processor.*

**Extends** OperationSink.

`dequeue(): T | undefined`

The oldest item, removed; undefined when the queue is empty.

### OperationSource`<TOp extends Operation>` extends `EventEmitter<{ outgoing: [TOp] }>` dataModel.ts

*Gives out the local operations a processor has stamped, for sending to collaborators.*

**Extends** EventEmitter (page 4). **Used by** MessageProcessor.

### MessageProcessor`<TR extends DataModelReader, TOp extends Operation>` extends `OperationSource<TOp>` dataModel.ts

*Applies operations to a data model: the only thing that changes it.*

**Extends** OperationSource. **Used by** ShapeMessageProcessor (page 6).

`readonly clientId: string`

The client whose local operations this processor applies.

`readonly model: TR`

The model's read side.

`readonly local: OperationSink<OperationContent<TOp>>`

Where the infrastructure puts the operations controllers send.

**MessageProcessor, continued**

`readonly remote: OperationSink<TOp>`

Where the collaboration protocol puts sequenced operations.

`process(): Promise<void>`

Applies the queued operations, interleaving local and remote, and emits the model's change events.

## Types

```
type OperationContent<T extends Operation> = T extends
   Operation ? Omit<T, Exclude<keyof Operation, "type">>
   : never
```

What a controller forms: an operation without the fields the processor stamps on it.

**Used by** MessageProcessor, LocalSend.

```
type LocalSend<TOp extends Operation> = (content:
   OperationContent<TOp>) => void
```

How a controller changes a model: it sends each local operation it forms to the message infrastructure, which enqueues it on the processor's local queue before returning, and later calls process.

# Interface summary 

31 interfaces and 10 type aliases from 10 files, in 7 groups of related interfaces. Inherited members are listed under the interface that declares them. Generated 2026-11-11 by npm run summary.

## ShapeCollectionWriter group

Refers to ShapeLocatorQueries, Shape, ShapeId (page 2); DataModelReader, MessageProcessor (page 5).

### DataModel dataModel.ts

*The write side of a data model: modification methods in derived interfaces.*

**Used by** ShapeCollectionWriter.

### ShapeCollection<S extends Shape> extends DataModelReader<ShapeCollectionEvents>, ShapeLocatorQueries shapeModel.ts

*Shapes in z-order, found by location.*

**Extends** ShapeLocatorQueries (page 2); DataModelReader (page 5). **Used by** ShapeCollectionWriter, ShapeMessageProcessor; ShapeView (page 4).

`get(id: ShapeId): S | undefined`
The shape with this ID, or undefined if no such shape exists.

`readonly size: number`
The number of shapes.

`forEach(visit: (id: ShapeId, shape: S) => void): void`
Calls visit for each shape, bottom to top. visit must not change the collection.

`compareZ(a: ShapeId, b: ShapeId): number`
Negative if a is below b, positive if above.

### ShapeCollectionWriter<S extends Shape> extends ShapeCollection<S>, DataModel shapeModel.ts

*The write side.*

**Extends** ShapeCollection, DataModel.

`add(id: ShapeId, shape: S, below?: ShapeId): void`
Adds shape under id: directly below the shape below, or on top when below is undefined.

`remove(ids: readonly ShapeId[]): number`
Removes every listed shape that is present; repeats and absent IDs are ignored.

### ShapeMessageProcessor<S extends Shape> extends MessageProcessor<ShapeCollection<S>, ShapeOp> shapeModel.ts

*Applies ShapeOps to a ShapeCollection through its writer.*

**Extends** MessageProcessor (page 5).

`newShapeId(): ShapeId`
An ID for a new shape, for an AddShape this client makes.

## Types

`type ShapeCollectionEvents = { change: [] }`
**Used by** ShapeCollection.

`type ShapeOp = AddShape | RemoveShapes | ClearShapes`
**Used by** ShapeMessageProcessor.